

CS184/284A Fall 2026 Homework 2 Write-Up

Carter Capetz and Charlie Herbst

<https://cal-cs184-student.github.io/hw2-charter-writeup/>

[Public write-up repository](#)

Overview

This homework took us from interpolating a few points to changing the connectivity of a whole triangle mesh. We implemented de Casteljau evaluation for Bézier curves and patches, area-weighted vertex normals, edge flips, edge splits, and Loop subdivision. The most useful lesson was how much bookkeeping lives underneath a simple-looking edit: changing one diagonal means keeping halfedges, twins, faces, vertices, and representative pointers consistent.

Section I: Bezier Curves and Surfaces

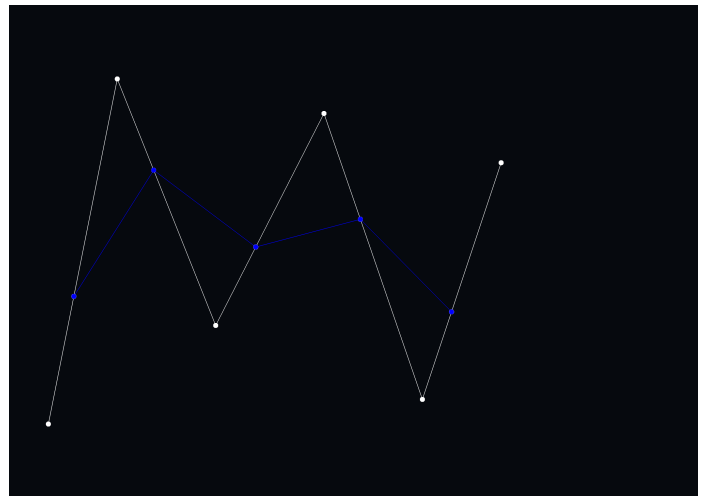
Part 1: Bezier curves with 1D de Casteljau subdivision

For each adjacent pair of control points, we calculate $(1 - t)P_i + tP_{i+1}$. That makes a shorter row of points. Our `evaluateStep` function returns just this next row; repeating the step eventually leaves one point on the curve. The viewer lets us step through each row with `E`; the green trace shows the full curve when `C` is on.

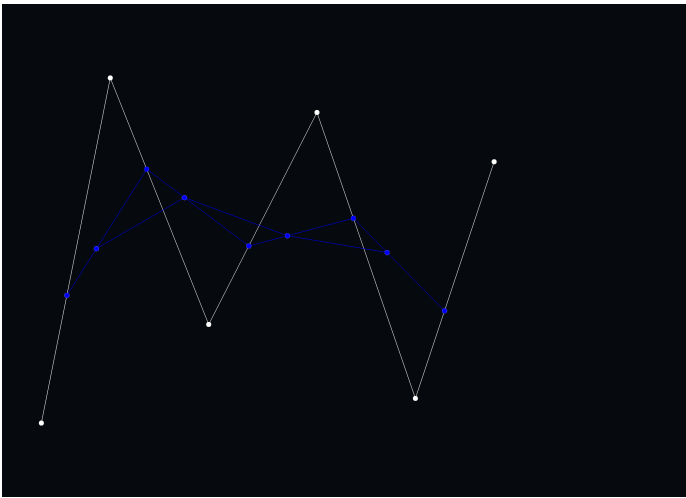
We made a six-point curve with a rise, a dip, and another rise so the intermediate levels are easy to follow. We saved it as `bzc/six_points.bzc`. The first sequence evaluates it at $t = 0.37$; each image adds one level, ending at the evaluated point.



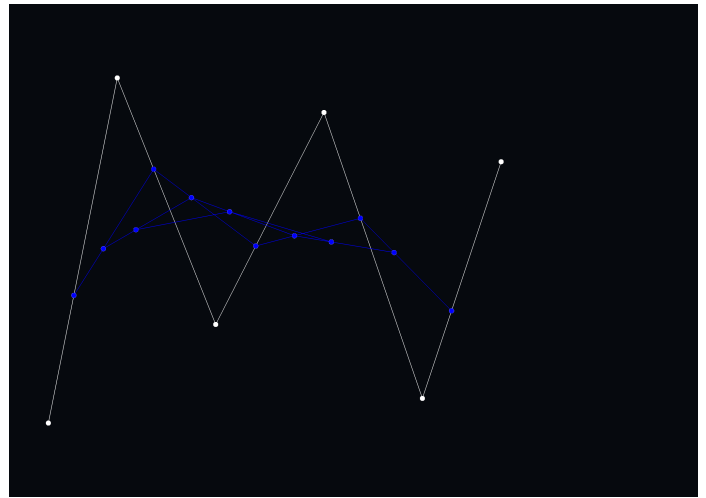
Original six control points.



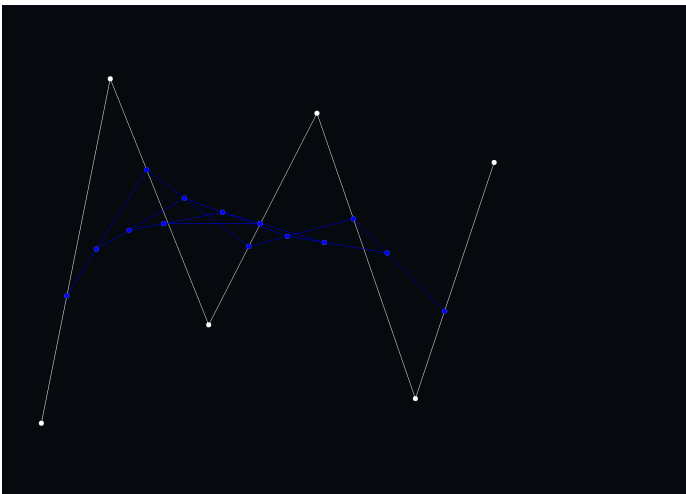
Level 1: five interpolated points.



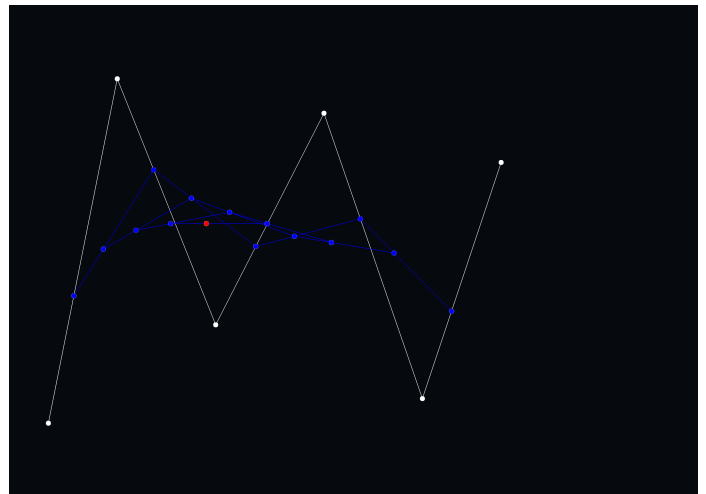
Level 2: four points.



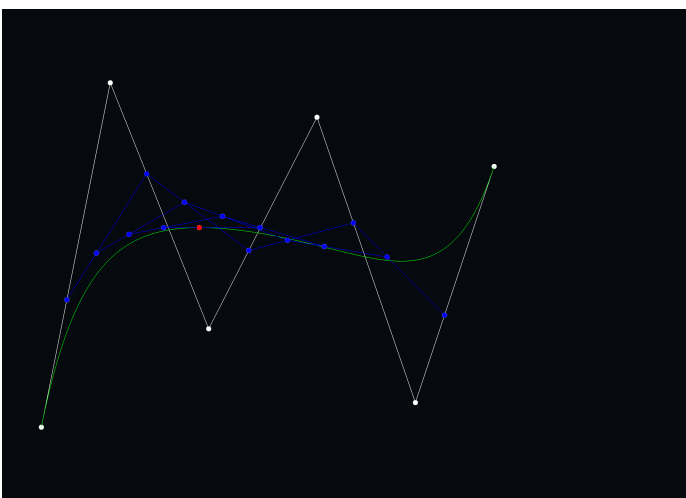
Level 3: three points.



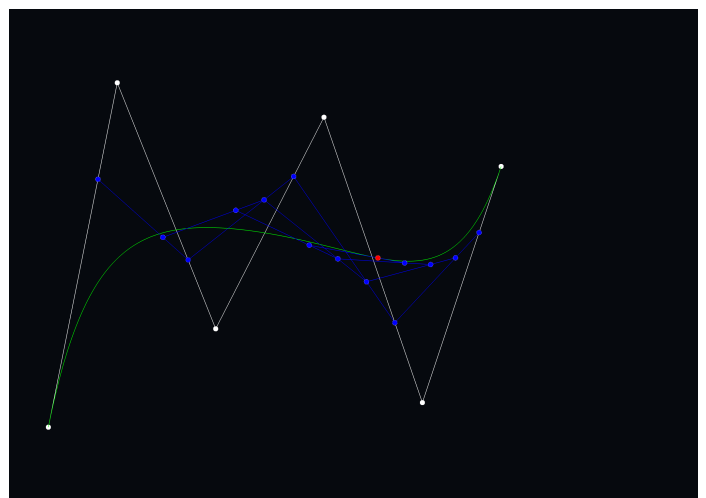
Level 4: two points.



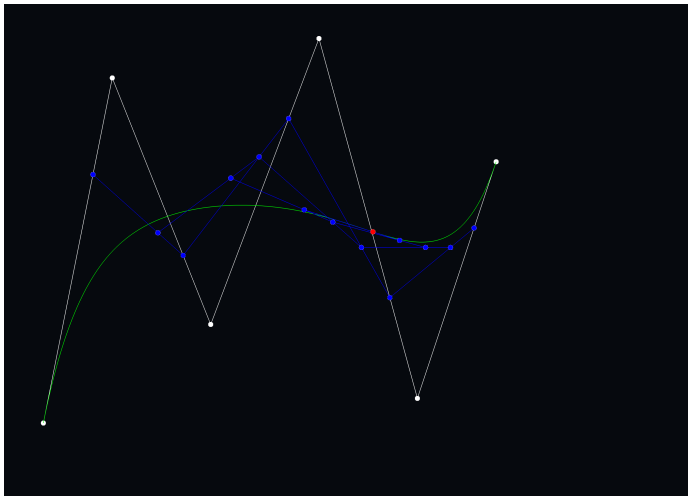
Level 5: the final point.



The complete curve traced in green.



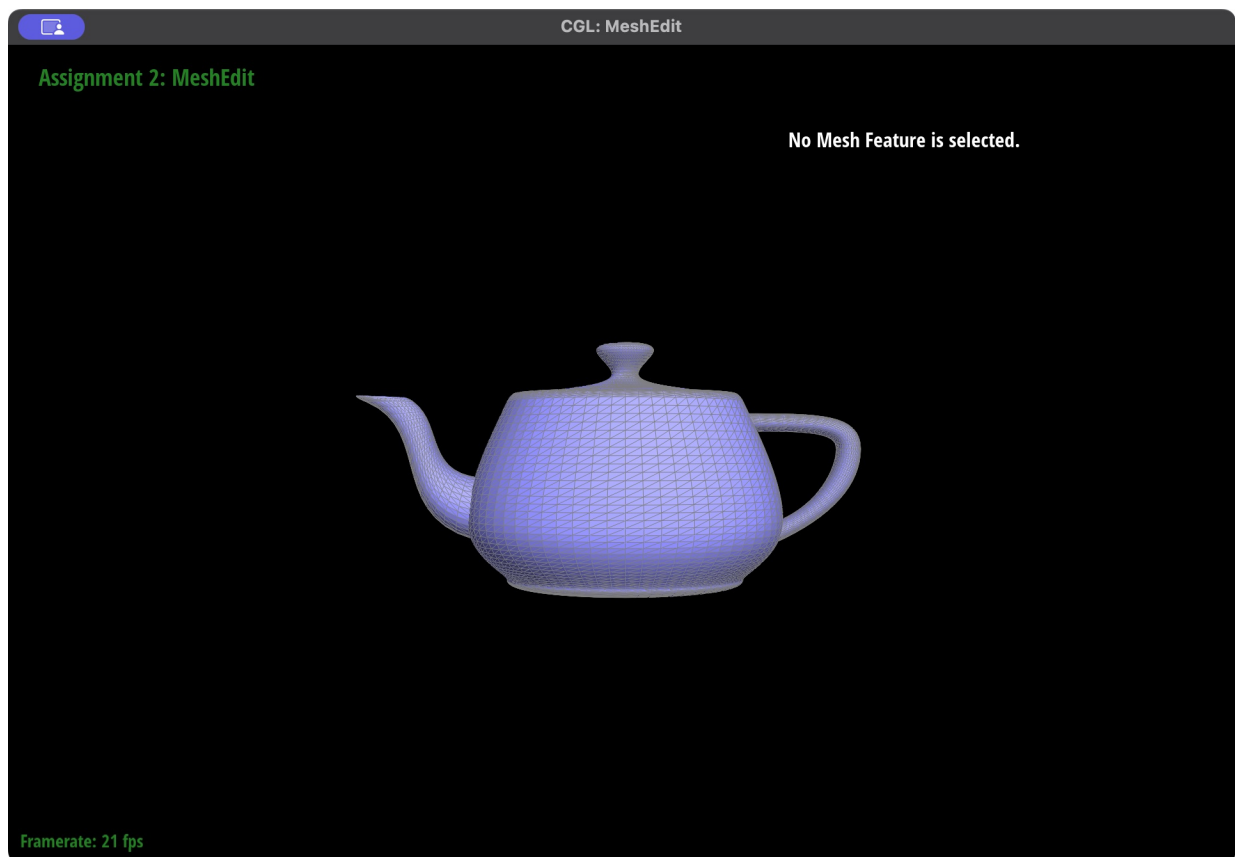
With $t = 0.72$, the evaluated point moves further along the curve.



We raised the fourth control point and changed t to 0.72 .
Both the curve and the evaluated point change.

Part 2: Bézier surfaces with separable 1D de Casteljau

For a surface, we apply the same interpolation to 3D points in two passes. The `evaluate1D` helper calls `evaluateStep` until one point remains. We use it to evaluate each row of control points at u , which gives a new column of points, then evaluate that column at v . This separable approach returns one position on the patch for each (u, v) . The teapot below comes from `bez/teapot.bez`.

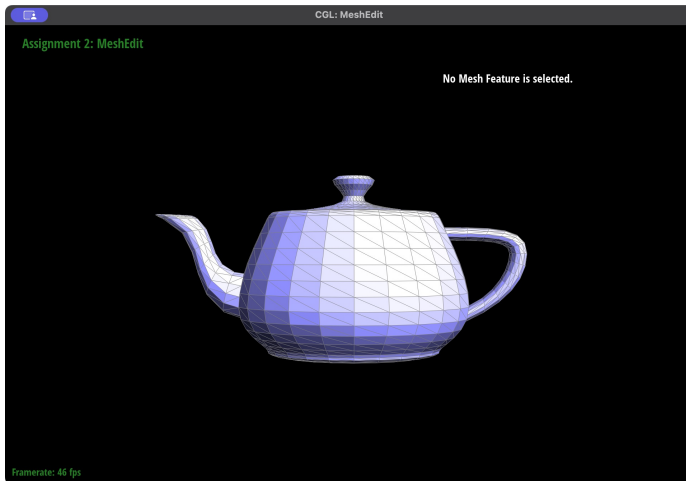


Bézier patch teapot produced by the surface evaluator.

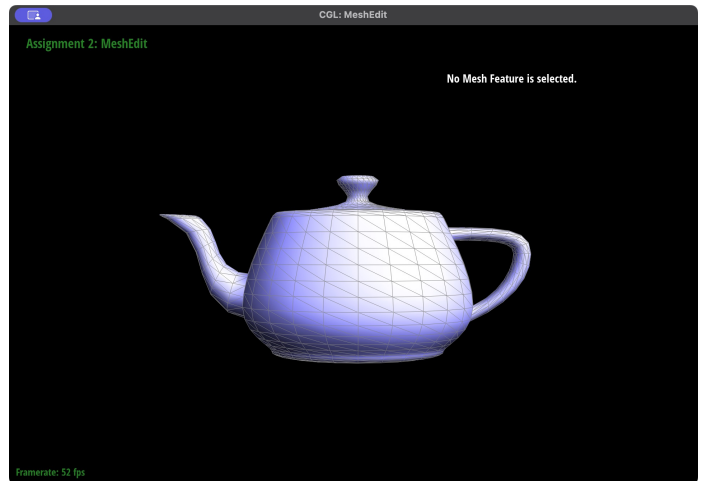
Section II: Triangle Meshes and Half-Edge Data Structure

Part 3: Area-weighted vertex normals

At each vertex, we walk the incident faces through the halfedge ring using $h = h \rightarrow \text{twin}() \rightarrow \text{next}()$, skipping virtual boundary faces. For each triangle, the cross product of its two outgoing side vectors points along its normal and has magnitude twice its area. We sum these unnormalized vectors, so larger triangles contribute more, then normalize once at the end. The common factor of two cancels during normalization. The result is the vertex normal used for smooth shading.



dae/teapot.dae with flat shading.

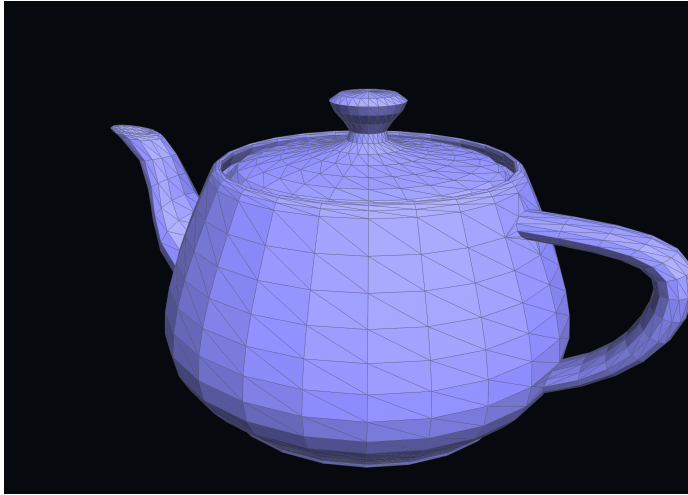


The same mesh with interpolated vertex normals. The facets blend much more smoothly.

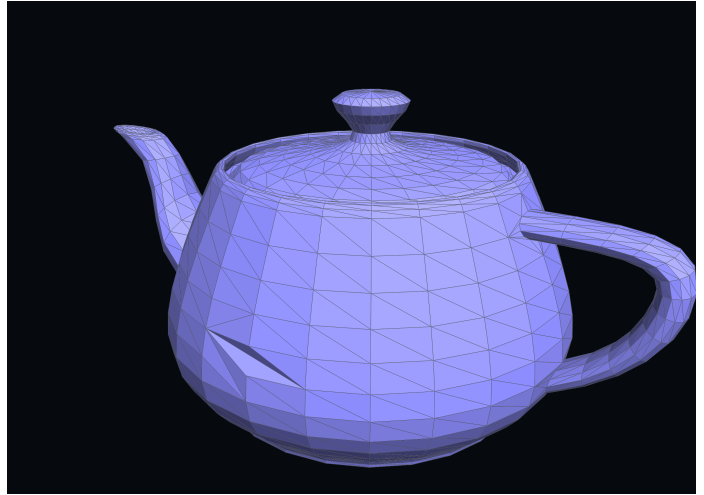
Part 4: Edge flip

An interior edge belongs to two triangles, which together form a quadrilateral. A flip replaces their shared diagonal with the other diagonal. We gather all six local halfedges and four vertices first, then reassign the six next, twin, vertex, edge, and face relationships. We also update the face and vertex representative halfedges. The operation reuses the same elements; it does not need new mesh objects.

We checked each face cycle and representative halfedge after repeated flips. A vertex's representative must still begin at that vertex, and each twin pair must describe the same edge in opposite directions. Boundary edges remain unchanged because they do not have a second real triangle to flip into.



Close-up of the teapot before three interior edge flips.

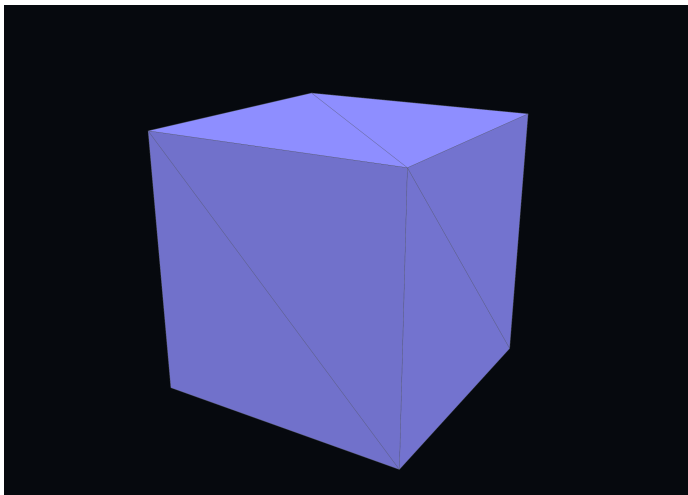


After three flips near the front of the teapot. The changed diagonals are local; the number of mesh elements stays the same.

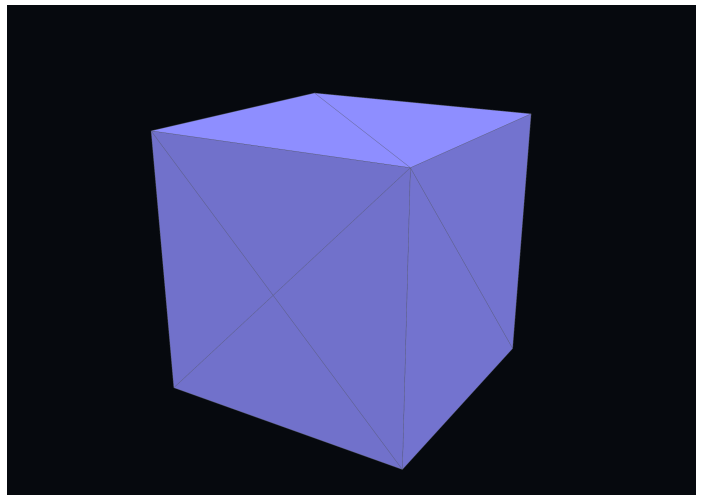
Part 5: Edge split

A split inserts a vertex at the selected edge's midpoint. The two adjacent triangles become four. We keep the original edge for one half of the old diagonal, create three more edges and two more faces, then wire the six new halfedges into the four triangle cycles. We mark the midpoint vertex as new. For subdivision, both halves of the original edge keep `isNew = false`, while the two edges joining the midpoint to the opposite vertices get `isNew = true`. This keeps the later flip pass from flipping either half of an original edge.

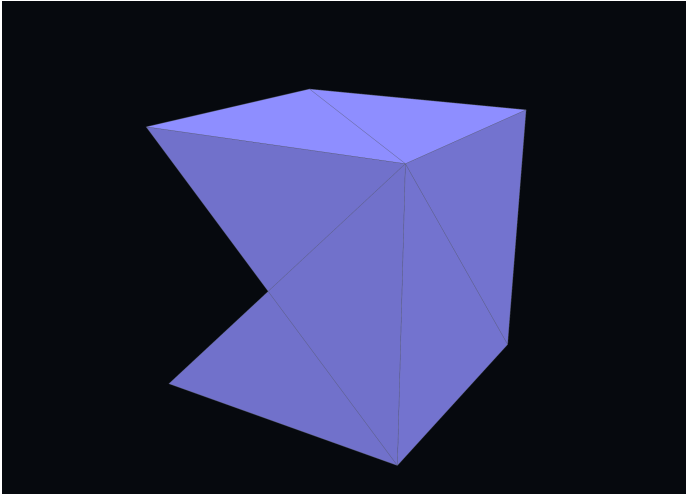
The cube face makes the change easy to see: one diagonal becomes a midpoint connected to the other two corners. We then flipped a neighboring edge across a cube crease to check that the operations work together. Choosing that edge avoids the collinear triangle that can result from immediately flipping a spoke in a flat, newly split face. Because the neighboring triangles lie on different planes, that flip changes the surface shape as well as its connectivity. Our checks verify face cycles, twin pairs, representative pointers, and the expected element counts after each edit.



Before: a face with one diagonal.



After: a midpoint and four triangles.



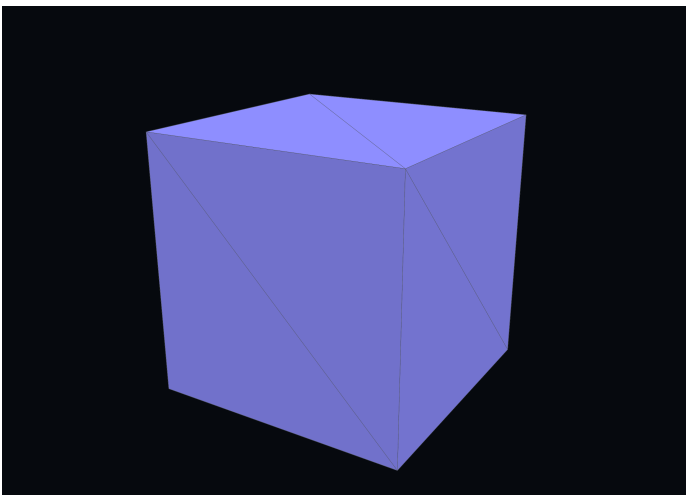
After the split and a neighboring edge flip across a crease.

We also implemented boundary splits for extra credit. We split only the real triangle into two, adding one vertex, two edges, one face, and four halfedges. On the virtual boundary face, the walk $B \rightarrow A$ becomes $B \rightarrow M \rightarrow A$; the loop stays closed. The midpoint's representative still lies on a segment of the original edge. We handle either orientation of the original edge's representative.

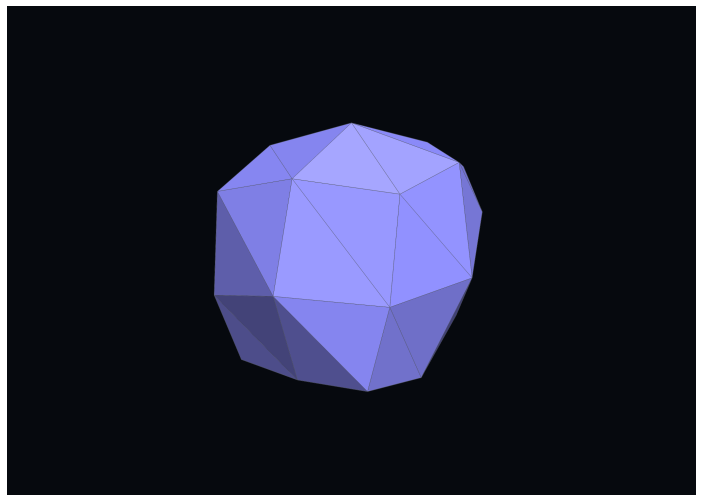
Part 6: Loop subdivision for mesh upsampling

We first record the original edges and compute all new positions from the untouched mesh. For an old vertex of degree n , each neighbor gets weight u , where $u = 3/16$ when $n = 3$ and otherwise $u = 3/(8n)$. The old vertex keeps weight $1 - nu$. A new edge point gets $3/8$ from each endpoint and $1/8$ from each of the two opposite vertices.

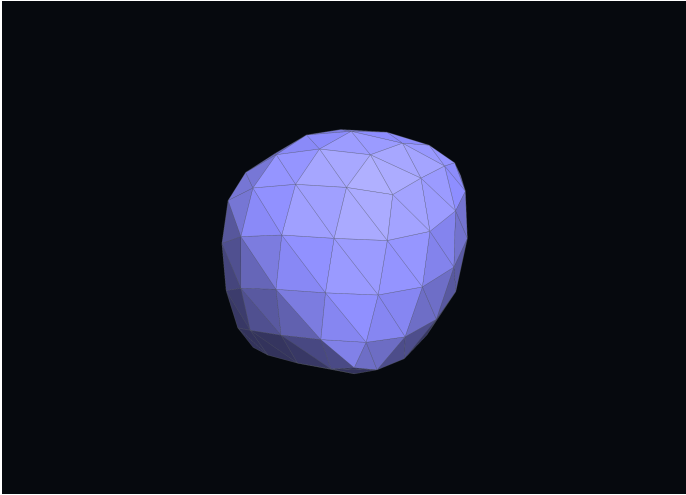
Only after saving those positions do we split every original edge. Then we flip new edges that connect one old and one new vertex, and finally assign the saved positions. Keeping the geometry update until the end matters: otherwise an early move would affect later weights.



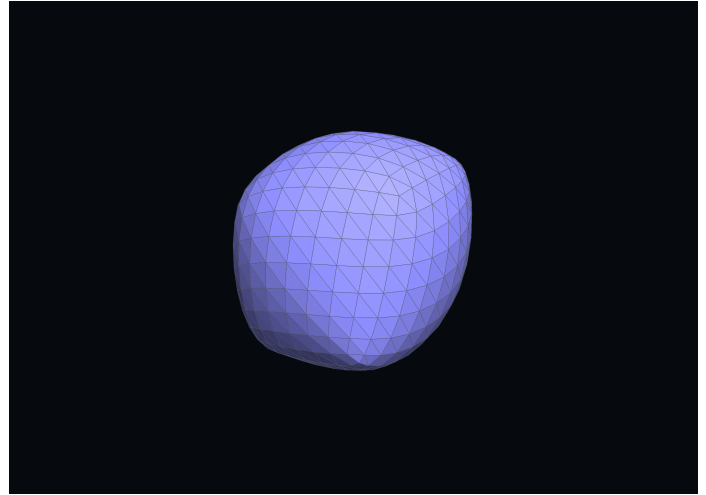
Original cube.



One Loop iteration.



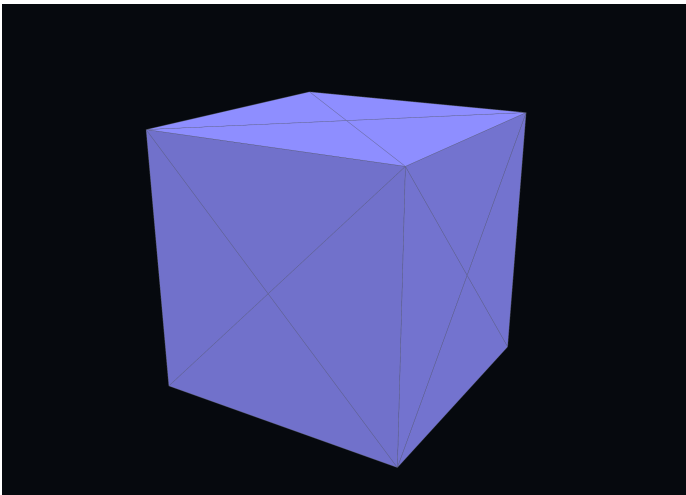
Two iterations.



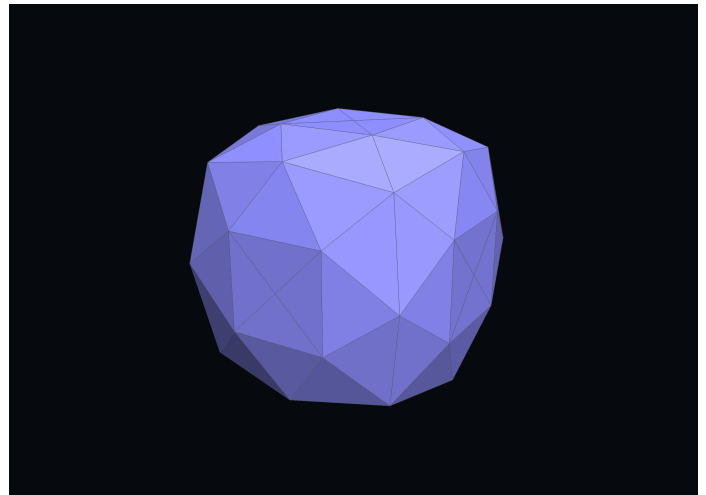
Three iterations. The formerly sharp corners are noticeably rounded.

Each iteration smooths sharp edges and corners because old vertices move toward their neighbors and the new edge points average nearby positions. The original cube has an uneven triangulation: each face diagonal gives its endpoints an extra neighbor. Corners therefore have different degrees and neighbor directions, so repeated averaging makes the result asymmetric.

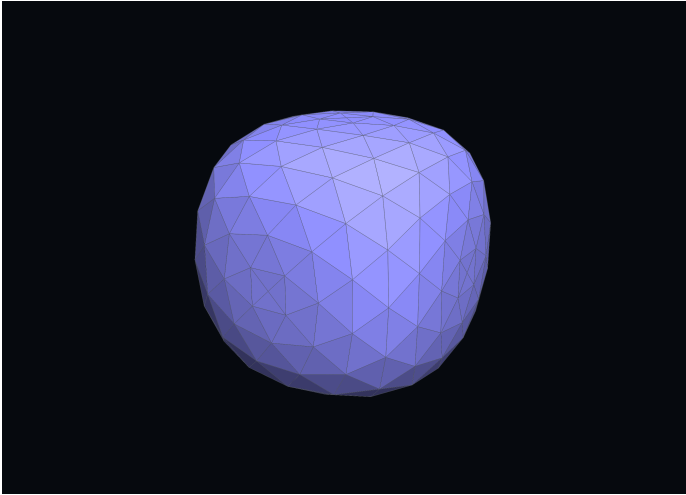
Flipping one face diagonal changes which corners receive that influence but does not remove the imbalance. To make the cube subdivide symmetrically, we instead split all six face diagonals. Each face then has a center connected to its four corners. All eight original corners have degree six with equivalent neighborhoods, and all six face centers have degree four. Loop treats corresponding vertices the same, so the symmetry survives repeated subdivision.



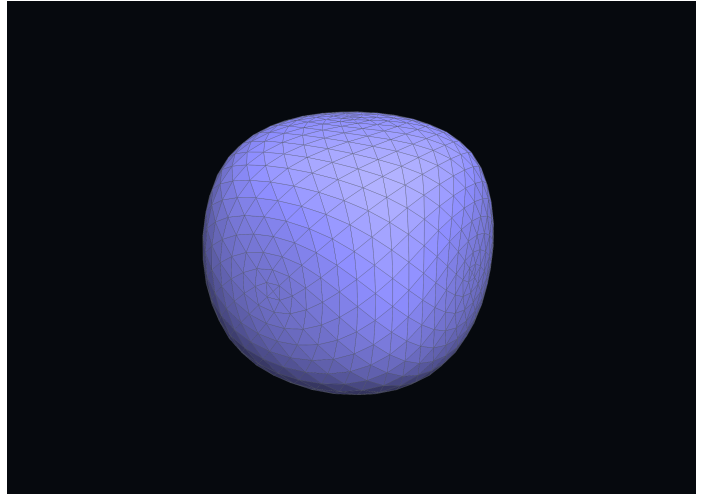
Preprocessed cube: 14 vertices and 24 triangles.



One Loop iteration after preprocessing.

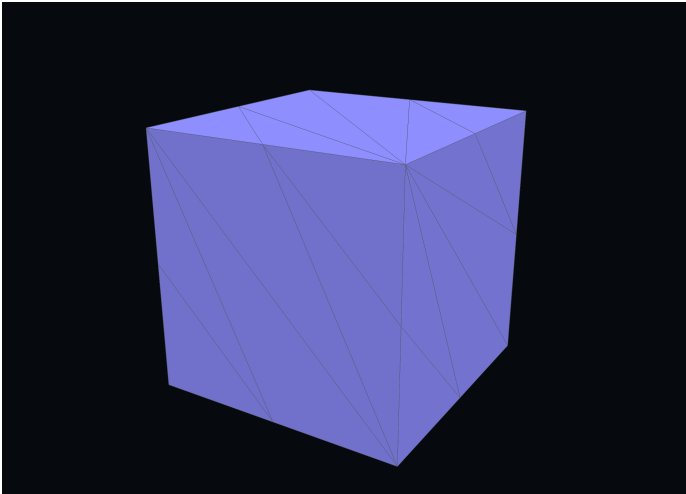


Two iterations; corresponding corners round evenly.

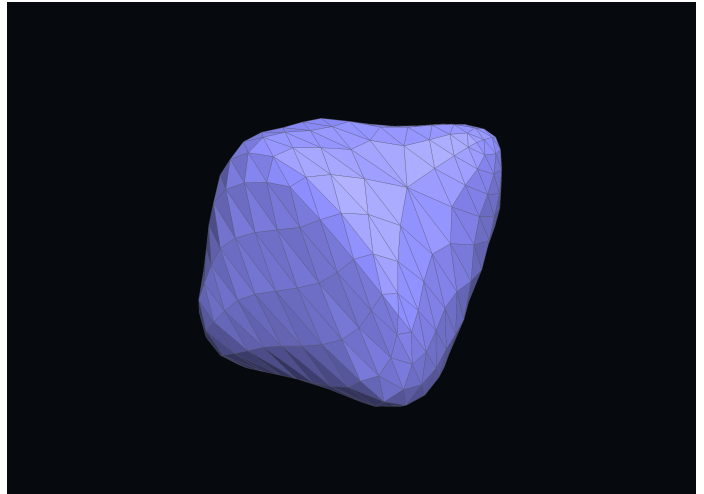


Three iterations, with the same camera as the original cube sequence.

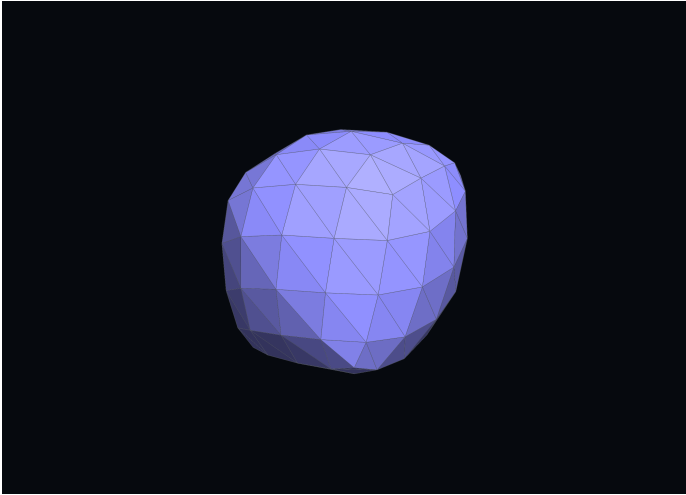
We separately tested pre-splitting the cube's twelve physical edges, where two faces meet at a right angle. This puts a midpoint closer to each corner along those edges. The comparison below uses two Loop iterations in both cases. The added samples reduce the rounding and shrinkage in this example, but do not create a permanent crease: repeated subdivision still smooths the feature.



Support experiment: the twelve physical edges are split once.



Supported cube after two iterations.



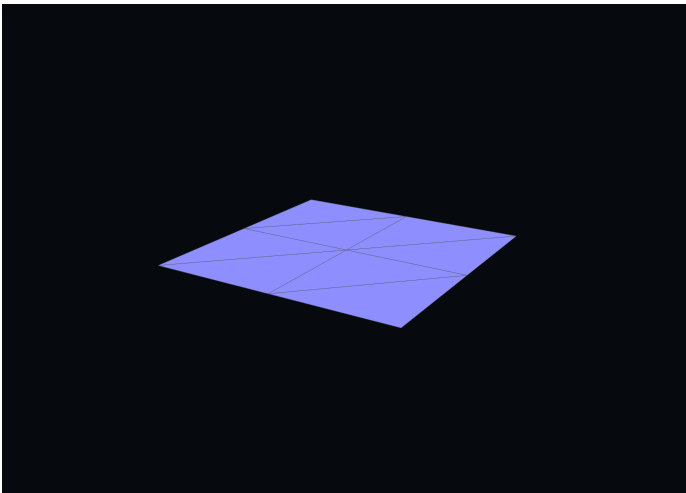
Original triangulation after the same two iterations.

Extra Credit

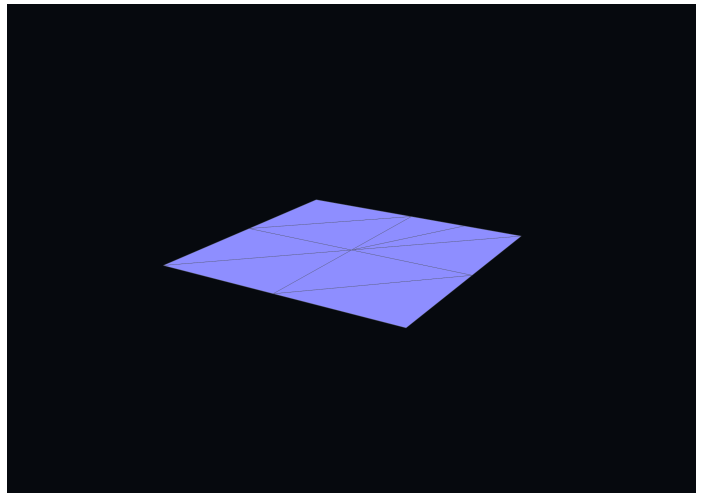
Boundary splits and boundary-aware Loop subdivision

For an edge on the boundary, the new vertex stays at its midpoint rather than using two opposite vertices. An old boundary vertex moves to $\frac{3}{4} P + \frac{1}{8} N_1 + \frac{1}{8} N_2$, where N_1 and N_2 are its two neighbors along the boundary. Interior vertices keep the usual Loop rule. We calculate every position before splitting edges, and boundary edges are never flipped.

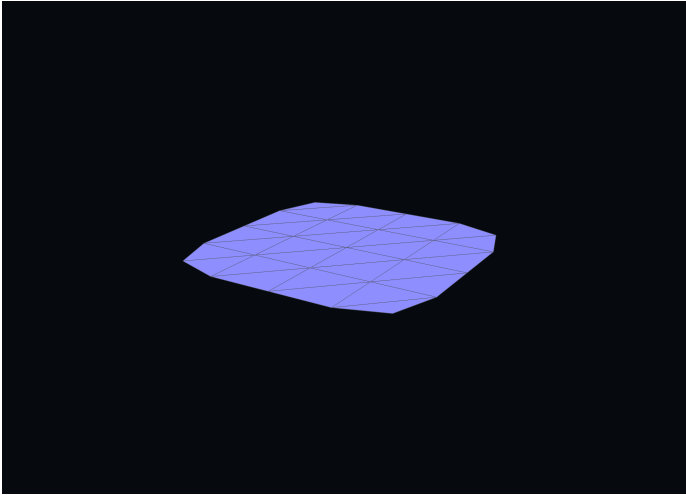
Our open patch has a real interior vertex and one outer boundary loop. The screenshots show a boundary split and two subdivision rounds. We also checked an annulus with two separate boundary loops. Those tests exposed a starter-code omission: a virtual boundary face did not have a representative halfedge. We set that pointer when constructing the loop and keep it valid after a split.



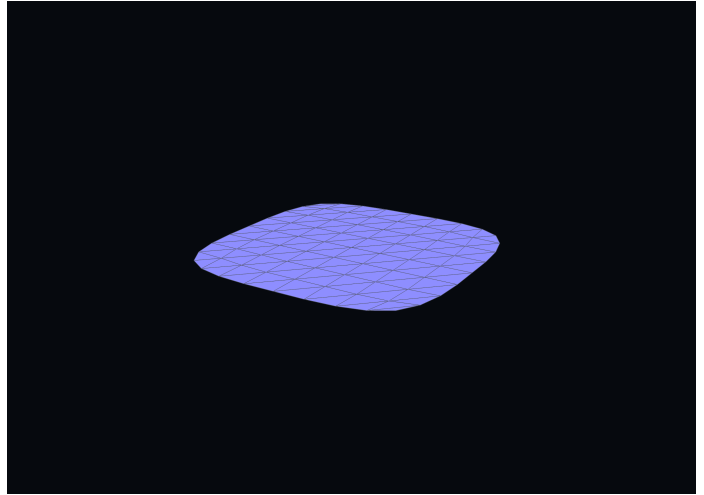
Open patch before editing: 9 vertices, 8 triangles.



One boundary split: 10 vertices, 9 triangles. The outer loop is still closed.



One Loop iteration: 25 vertices, 32 triangles.

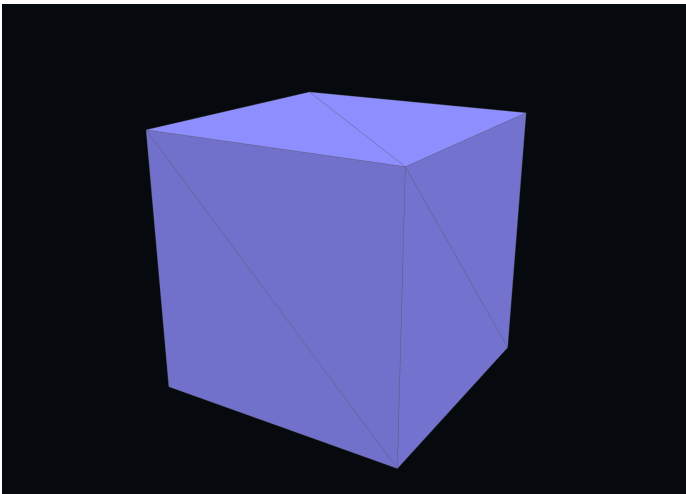


Two iterations: 81 vertices, 128 triangles. Boundary corners round without closing the opening.

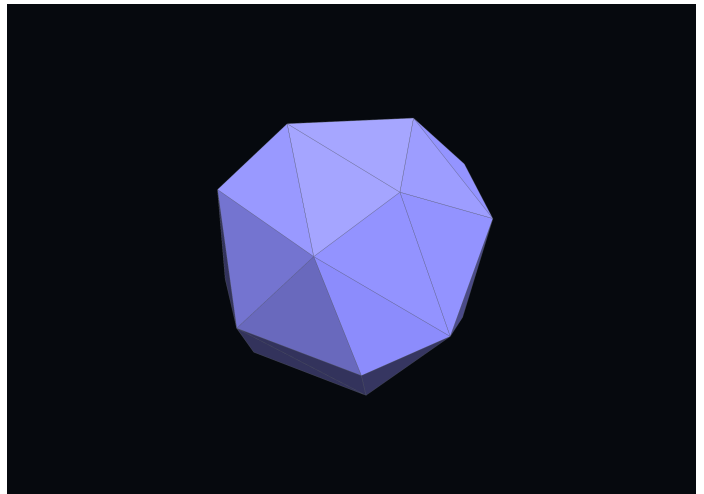
An additional scheme: $\sqrt{3}$ subdivision

We implemented [Kobbelt's \$\sqrt{3}\$ subdivision](#) for closed triangle meshes. Press U to use it; L still runs Loop. We insert a centroid into each original face, then replace each original edge with a connection between the centroids on its two sides. Each face becomes three triangles, so the face count grows by three instead of Loop's factor of four.

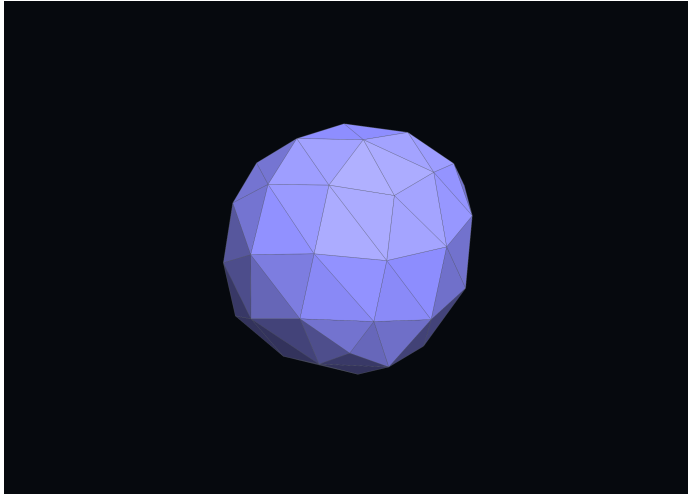
For an original vertex of degree n , we use $\alpha = (4 - 2 \cos(2\pi/n))/9$ and update it to $(1 - \alpha)P + (\alpha/n)\Sigma N$. Centroids use the original face positions. We build the resulting triangles from saved vertex and face indices, which expresses the centroid split and original-edge flip without introducing another local pointer-rewiring routine. This rebuild invalidates old selections, so the viewer clears them afterward. This scheme supports closed meshes; open meshes use our boundary-aware Loop implementation.



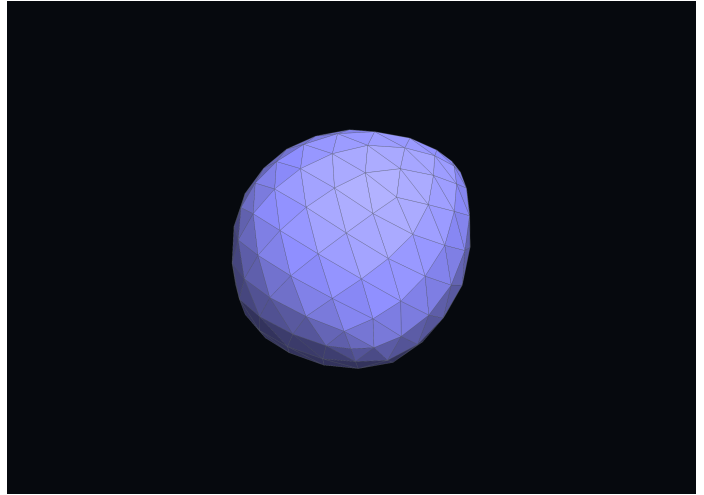
Original cube: 12 triangles.



One $\sqrt{3}$ iteration: 36 triangles.



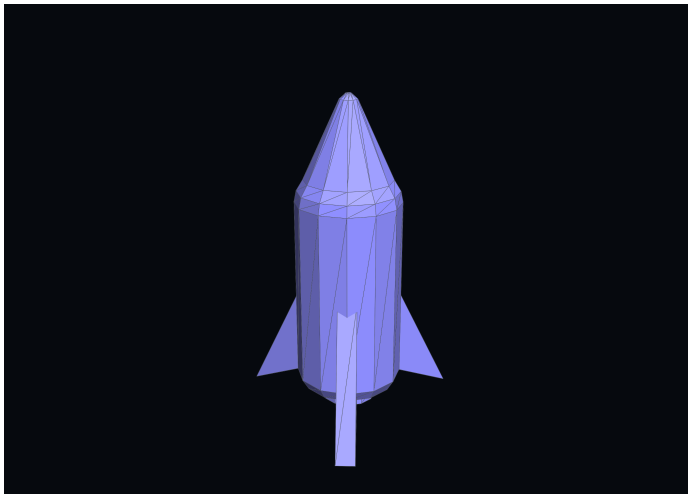
Two iterations: 108 triangles.



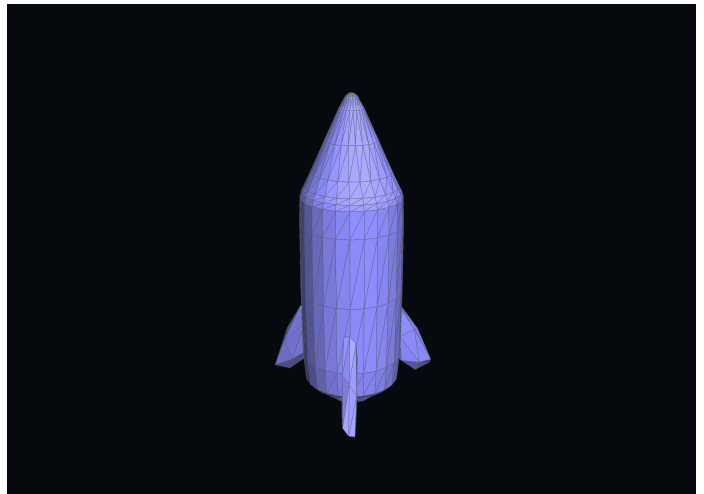
Three iterations: 324 triangles. The connectivity differs from Loop's midpoint grid.

Section III: Creative Mesh and Custom Shader

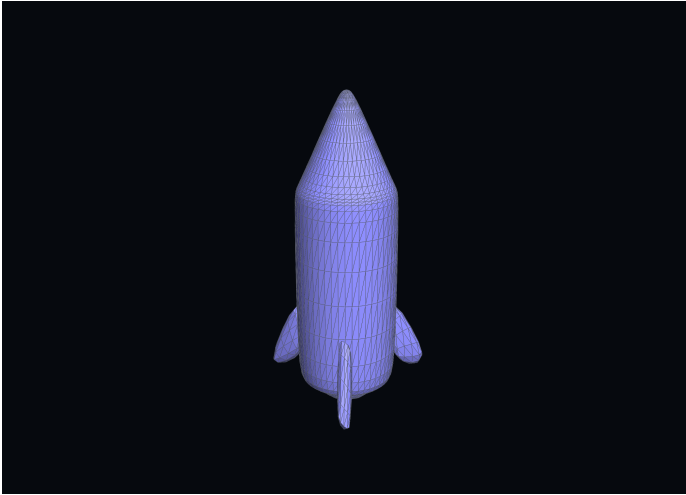
We designed a retro rocket instead of the tutorial's humanoid. We generated twelve-sided rings with changing radii for the nozzle, body, shoulder, and nose, capped both ends, and added three thick triangular fins. Closely spaced rings near the nozzle and shoulder support those features during smoothing. The fuselage and fins are separate closed components that overlap at the roots; they do not share nonmanifold edges. The control mesh has 128 vertices and 240 triangles and is saved as [artcompetition.dae](#).



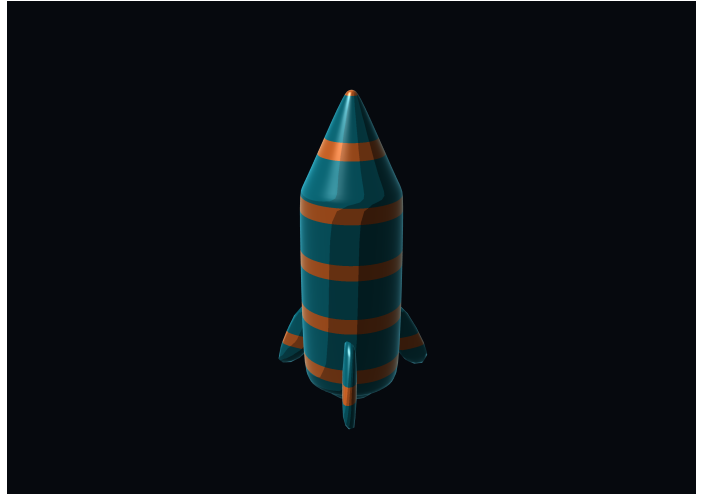
Our original rocket control mesh.



One Loop iteration: 960 triangles.



Two iterations: 3,840 triangles. The body and fins become smoother.



Two iterations with our custom shader.

Our custom fragment shader mixes teal paint with copper bands based on the surface's height. Four diffuse-light levels give it a slightly illustrated appearance, while a pale rim term outlines surfaces facing away from the eye. A narrow specular term adds highlights. In MeshEdit, press Q for smooth normals, W for shader mode, and 2 for this shader. The bands are procedural, so they do not require texture coordinates.

We checked triangle cycles, twins, representative pointers, normals, and subdivision positions after repeated edits. Report images for the new experiments were captured from the actual MeshEdit framebuffer with fixed camera settings. This keeps comparisons consistent and makes them reproducible.